

MAE3185 Project Memo

Name: Nicholas Rios

Section Number: Section 002

Partner(s): Jacob Mai

Report Date: 11/25/2025

Equipment Setup:

MAE Supplied Kit with a touchscreen, touchscreen adapter, solderless protoboard, wires, a bluetooth module, a steel ball, two rod ending spherical bearings, and a Raspberry Pi Pico H. Two 20-kg servos were used as well, however they were not provided in the kit.

Objectives:

Develop a mechatronic system capable of balancing a ball in a specified location of rectangular area, while the ball is subject to external forces and impulses. Objectives that must be completed prior to the main objective consist of developing a mechanical system, electrical layout, and PID coding loop that will all support the main objective of balancing the ball.

Results:

At the conclusion of the allotted work term, the main goal of developing a balancing ball system was completed successfully. The final system's construction had an efficient mechanical design, an intuitive electrical layout, and a PID code that effectively maintained the ball's position despite input forces being applied to the ball.

Conclusions:

The main conclusions of the project are as follows:

1. Crank-slider is an effective mechanical choice for servo control of the touchscreen area.
2. PID control loops within micropython are a simple and effective tool for creating a closed loop feedback, thus driving up efficiency and accuracy of ball placement.

Trial and error of the mechanical design led to a crank-slider being the simplest and most fortified way to control the axes of the touchscreen area using only two motors. The crank-slider drives the motion of an attached linear actuator, which rotates the touchscreen along either axis. The main drawback of this design is the applied torque on the servo, which led to a necessary upgrade from 3-kg servo to a 20-kg servo. The crank-slider mechanism perfectly transforms the rotational ability of the servo into linear actuation. This linear actuation allows for a high precision control of the "x" and "y" axes of the touchscreen area. Using the rod ending spherical bearings allowed for a linear actuator to be directly attached to the side of the touchscreen surface with no worry of extraneous torques being placed on the system. Previous iterations show that the actuators were attached by ball joints, which can be seen in Figure 1. This design was abandoned due to implementation complications, as the supplied ball joints were significantly easier to implement. A more finalized design can be seen in Figure 2. In conclusion, all improvements made to the mechanical design

MAE3185 Project Memo

resulted in a system strong enough to control the position of steel balls with varying weights. For visual representation of the functioning system, see Figure 3.

Aside from the mechanical design, another important development was the code to control the system. Implementing a PID control loop within MicroPython was a simple yet highly effective tool for establishing a closed-loop feedback. It continuously calculated error and adjusted the servo output in realtime to transform the raw sensor data into a smooth and stable motion. In doing this it maximized the accuracy of the ball's placement and the efficiency of the mechanical response. To see reference points of the code's development, see Figures 4 and 5. These images highlight important snippets of the overall code and contain comments on what that piece of code does. In the end, it proved that lightweight control algorithms are sufficient for achieving precise physical stabilization without over complicated hardware.

Figures:

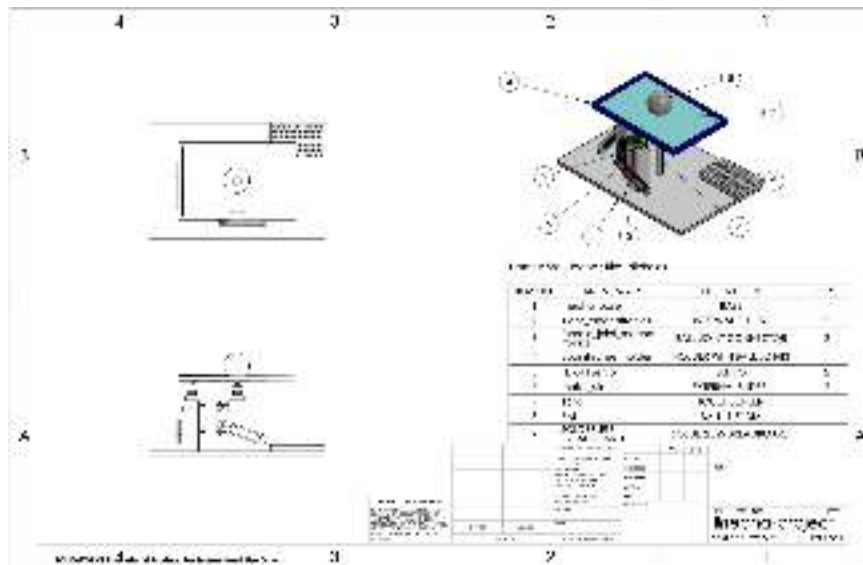


Figure 1. Initial Design

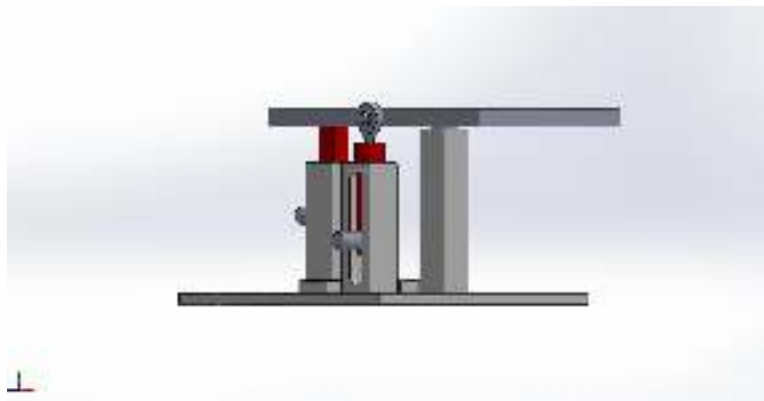


Figure 2. Improved Design



Figure 3. Final System Iteration

Sample Code Snippets:

```
class PID:
    def __init__(self, kp, ki, kd, i_limit=20.0, out_min=-50.0, out_max=50.0):
        self.kp, self.ki, self.kd = kp, ki, kd
        self.i_sum = 0.0
        self.prev_err = 0.0
        self.i_limit = i_limit
        self.out_min = out_min
        self.out_max = out_max
    def reset(self):
        self.i_sum = 0.0
        self.prev_err = 0.0
    def step(self, err, dt):
        #Integral term to prevent the system from overshooting
        #aggressively if the ball is in one spot for too long
        self.i_sum += err * dt
        if self.i_sum > self.i_limit: self.i_sum = self.i_limit
        elif self.i_sum < -self.i_limit: self.i_sum = -self.i_limit
        #Derivative term to calculate the rate of change and
        #predict future movement. it handled division by zero
        #if dt is negligible
        d = (err - self.prev_err) / dt if dt > 0 else 0.0
        self.prev_err = err
        #The whole PID calculation based on general form of
        #Output = Kp*e + Ki*integral + Kd*derivative
        out = self.kp * err + self.ki * self.i_sum + self.kd * d

        if out > self.out_max: out = self.out_max
        elif out < self.out_min: out = self.out_min
        return out
```

Figure 4. PID snippet

```
#This was the filter used to smooth out all the
#jitters from the servo
xf = EMA_ALPHA_X * x + (1 - EMA_ALPHA_X) * xf
yf = EMA_ALPHA_Y * y + (1 - EMA_ALPHA_Y) * yf

#Converts pixels to -1 to 1 range, making 0 the center
#center. This is so the PID math is independant of the
#screen resolution.
ex = (xf - X_MID) / (X_SPAN / 2)
#Uses Y_SCALE because the screen is not a perfect square,
#so the Y_SCALE corrects it to feel and move consistant
#in both axis.
ey = (yf - Y_MID) / (Y_SPAN / 2) * Y_SCALE
```

Figure 5. Filtering and scaling snippet